End-to-End Cervical Cancer Risk Prediction Pipeline

1. Introduction

This project implements an end-to-end machine learning system for cervical cancer biopsy risk prediction using the UCI Cervical Cancer (Risk Factors) dataset. The objective is not to optimize model performance, but to demonstrate a complete and reproducible machine learning workflow that includes data ingestion, preprocessing, feature engineering, model training, inference, prediction storage, and data drift monitoring.

The system is designed to mimic key components of a production-oriented ML pipeline, rather than serving as a one-time modelling experiment. Since the original dataset is static, the project simulates a clinical screening workflow by splitting the data into historical records for model development and future records that arrive as daily batches. This allows the pipeline to demonstrate how new data would be processed, how predictions would be generated and stored, and how incoming data would be monitored for distribution shifts over time.

The prediction task is to estimate whether a record has a positive biopsy result based on cervical cancer risk-factor variables. The resulting model is intended only as a technical demonstration of pipeline design and is not a clinically deployable risk model.

2. Data Source, Target, and Cadence

This project uses the Cervical Cancer (Risk Factors) dataset from the UCI Machine Learning Repository. The dataset contains 858 records and includes risk-factor variables related to age, behavioral and reproductive health-related factors, STD-related history, and previous diagnosis indicators. Although the dataset is static and does not represent a real data stream, it is still suitable for this assignment because it contains clinically relevant features and substantial missingness, which allows the pipeline to demonstrate preprocessing, missing value handling, and data-quality monitoring. Its small size also keeps the full pipeline fast to run and easy to reproduce on a local machine.

The dataset contains four target variables: Hinselmann, Schiller, Citology, and Biopsy. I selected Biopsy as the main prediction target because it is the most clinically interpretable endpoint among the four. Biopsy is a binary variable indicating whether the biopsy result is positive. The other three target variables are excluded from the feature set to avoid target leakage, because they represent additional screening or diagnostic outcomes that are likely to be highly related to the biopsy result rather than baseline risk-factor inputs.

One limitation of the dataset is that the positive biopsy class is rare. In the full dataset, 55 out of 858 records are biopsy-positive, corresponding to a positive rate of approximately 6.4%. Because of this class imbalance, the historical/future split was stratified by the Biopsy outcome to ensure that both subsets contained biopsy-positive cases. This stratification

creates a stable simulated incoming data stream rather than an intentionally shifted deployment scenario; therefore, an optional synthetic stress test is included later to verify that the drift monitoring logic can flag a data-quality shift when one occurs. The data was split into 70% historical records and 30% simulated future records. The historical subset contains 600 records with a positive biopsy rate of approximately 6.3% (38 out of 600), while the future subset contains 258 records with a positive biopsy rate of approximately 6.6% (17 out of 258).

Since the original dataset does not contain real timestamps or a natural data arrival process, I simulated a daily batch ingestion cadence. The 258 future records were randomly shuffled using a fixed random seed and divided into 14 approximately equal daily batches, with 18–19 records per day. Synthetic arrival dates were assigned from 2026-05-08 to 2026-05-21, both inclusive.

Daily batch ingestion was chosen because it is a realistic cadence for a screening workflow, where new records may be accumulated, checked, and processed periodically rather than streamed in real time. This design keeps the system simple to run on a local machine while still demonstrating how new data can be ingested, preprocessed, predicted, stored, and monitored over time.

3. Pipeline Overview

The project is organized as a modular end-to-end machine learning pipeline. In this context, modular means that the main functions are separated into reusable components. The main workflow starts from a static public dataset, creates a simulated daily data ingestion process, trains a model on historical records, and then applies the saved preprocessing and prediction pipeline to incoming daily batches. The pipeline also stores predictions and monitors incoming data for potential distribution shifts.

The main pipeline stages are:

1. **Data loading and batch simulation**
   The original UCI dataset is loaded from *data/external/*. Missing values encoded as '?' are converted to NaN, and a synthetic record_id is assigned because the original dataset does not contain patient identifiers. The data is then split into historical and future subsets. Future records are assigned synthetic arrival dates to simulate daily batches.
2. **Preprocessing and feature engineering**
   Numeric and binary features are processed separately, missingness indicators are added, and numeric features are scaled before logistic regression.
3. **Model training and evaluation**
   The model is trained only on the historical subset. Five-fold stratified cross-validation is used for model evaluation and model selection under class imbalance. The final model is a sparse logistic regression pipeline with L1-based feature selection, limited to at most 10 selected features, followed by a class-weighted L2 logistic regression classifier.
4. **Daily inference**

For each simulated daily batch, the saved preprocessing and prediction pipeline is loaded and applied to the new records. This means that missing value imputation, missingness indicator creation, numeric feature scaling, feature selection, and prediction are applied consistently to each incoming batch. The pipeline outputs biopsy-positive probabilities and predicted labels. Observed biopsy labels are kept only for offline evaluation in this demonstration and are not used during inference.

5. **Prediction storage**
   Prediction outputs are saved both as CSV files and in a SQLite database. CSV files provide easy inspection and reproducibility, while SQLite stores two structured operational tables: ingestion_log and predictions.
6. **Drift monitoring**
   Each incoming daily batch is compared with the historical baseline distribution. The pipeline checks missingness drift, numeric feature mean shifts, and prediction score shifts. An optional synthetic stress test is also included to verify that the monitoring logic can flag a data-quality shift when one is introduced.

The full workflow can be executed with *scripts/run_full_demo.py*, while individual steps can also be run separately through the training, inference, drift-checking, and stress-test scripts. Detailed running instructions are provided in the repository README.md.

4. Preprocessing and Feature Engineering

The original dataset encodes missing values as '?'. During data loading, these values are converted to NaN, and all feature columns are converted to numeric values. The target variables are removed from the feature matrix before preprocessing to avoid leakage. The selected target, Biopsy, is kept separately as the outcome variable.

Missing value handling is an important part of this project because several risk-factor and diagnosis-related variables contain missing values. The preprocessing pipeline handles numeric and binary features separately. Numeric features, such as age, duration variables, and count variables, are imputed using the median. Binary features, such as smoking status, and STD indicators, are imputed using the most frequent value.

In addition to imputation, missingness indicators are added. This is an important design choice because missingness itself may carry information. A missing value may reflect that information was not available, not recorded, or not applicable. By adding missingness indicators, the model can distinguish between an observed value and a value that was filled in during imputation.

After imputation, numeric features are scaled using standardization before logistic regression. Scaling is applied only to numeric features, while binary features are left as binary indicators after imputation. The same fitted preprocessing pipeline is also reused during daily inference, so incoming batches are processed consistently with the historical training data.

The final feature engineering step is L1-based feature selection. This step is applied after preprocessing and selects at most 10 features before the final classifier. Feature selection

was included to reduce redundancy among correlated variables, such as smoking status, smoking duration, and smoking intensity, and to keep the final model more compact and interpretable. The selected features are used for model inspection and should not be interpreted as causal risk factors.

5. Model Development and Selection

Model development was performed only on the historical subset. The simulated future records were not used for model selection, preprocessing fitting, or hyperparameter decisions. They were kept as incoming daily batches for inference and monitoring.

Because the biopsy-positive class is rare, accuracy was not used as the main evaluation metric. If accuracy were used as the main metric, a model that predicts all records as biopsy-negative would already achieve high accuracy but would fail to identify positive cases. Therefore, model evaluation focused on metrics that are more informative under class imbalance: ROC-AUC, average precision, recall, precision, F1-score, and the confusion matrix.

I used five-fold stratified cross-validation on the historical data. Stratification was used to keep the rare biopsy-positive class represented in each fold as much as possible. All preprocessing and feature selection steps were included inside the scikit-learn pipeline. This means that imputation values, scaling parameters, missingness indicator structure, feature selection, and model parameters were fitted only on the training folds and then applied to the validation folds. This avoids information leakage from the validation folds into preprocessing or model selection.

Several candidate models were evaluated during development. Logistic regression models were used as transparent baseline models, while tree-based models were included because they can capture nonlinear relationships and provide a comparison with less linear modelling assumptions. In the implementations used here, the tree-based models can also handle missing values, so they served as a useful sensitivity analysis for the explicit imputation strategy used in the logistic regression pipeline. For Random Forest and XGBoost, a small random search was used to select reasonable hyperparameters on the historical subset. The search was intentionally limited to avoid overfitting the small and imbalanced dataset.

Table 1. Mean five-fold cross-validation performance of candidate models

Model	**ROC-AUC**	**Average precision**	**Recall**	**Precision**	**F1-score**
Full logistic regression	0.558	0.138	0.350	0.094	0.146
Sparse logistic regression	**0.652**	0.157	**0.425**	**0.125**	**0.192**
Random Forest	0.604	0.146	0.239	0.109	0.147
XGBoost	0.589	**0.191**	0.214	0.065	0.098

The sparse logistic regression model was selected as the final model. Although XGBoost achieved the highest average precision, it had lower recall and F1-score and added extra dependency complexity. Random Forest also did not outperform the sparse logistic regression model. The sparse logistic regression pipeline provided the best balance of performance, stability, interpretability, and simplicity for this small and imbalanced dataset.

The final model consists of L1-based feature selection followed by a class-weighted L2 logistic regression classifier. The L1-based selector was limited to at most 10 features. This was included to reduce redundancy among correlated variables, such as smoking status, smoking duration, and smoking intensity, while keeping the model compact. The final classifier used class weighting to reduce the effect of biopsy class imbalance.

The selected features and logistic regression coefficients were inspected after fitting the final model on the full historical subset, as shown in the table below. Positive coefficients indicate association with higher predicted biopsy-positive risk in this fitted model, while negative coefficients indicate association with lower predicted risk. These coefficients are used only for model inspection. They should not be interpreted as causal effects or clinically validated risk factors.

Table 2. Selected features and coefficients from the final sparse logistic regression model

Feature	Feature Meaning	Coefficient
Missing (Hormonal Contraceptives (years))	Missing indicator for hormonal contraceptive use duration	-1.870
STDs:syphilis	STD: Previous syphilis diagnosis indicator	-1.750
Dx:HPV	Previous HPV diagnosis indicator	1.410
Dx:CIN	Previous CIN diagnosis indicator	1.400
STDs:HIV	STD: Previous HIV diagnosis indicator	0.813
Missing (STDs (number))	Missing indicator for number of STDs	0.795
IUD	IUD use indicator	0.768
Dx	General previous diagnosis indicator	0.738
Missing (STDs: Time since first diagnosis)	Missing indicator for time since first STD diagnosis	-0.309
Missing (IUD (years))	Missing indicator for IUD use duration	-0.308

The final model is therefore a transparent baseline model for demonstrating the pipeline. It is not intended to be a clinically optimized or externally validated risk prediction model.

6. Storage Design

This project uses both flat files and a lightweight SQLite database. The storage design separates generated files and outputs from operational records.

Flat files are used for generated files and outputs that should be easy to inspect, regenerate, and version. These include the original public dataset, the generated historical/future split files, daily prediction CSV exports, the trained model, evaluation reports, and drift reports. This makes the pipeline transparent and easy to debug, because each generated file can be opened and checked directly.

SQLite is used for structured operational records that need to be queried over time. In this project, the SQLite database is stored as *data/pipeline.db* and contains two tables: *ingestion_log* and *predictions*. Using a database here has several advantages over storing everything as separate CSV files. It allows efficient filtering and aggregation by batch date, record ID, and model version. For example, the system can quickly summarize how many records were processed or predicted as positive within a given time window. It also better resembles how operational prediction records may be queried in a screening system, where different users or downstream services may need to query the same records consistently. In addition, if biopsy outcomes become available after the initial prediction, the stored prediction records can be updated in a simple and fast way.

The *ingestion_log* table records one row per processed daily batch and stores batch-level metadata, as shown in the example below.

Table 3. Example record from the ingestion_log table

batch_date	n_records	n_positive_labels	positive_rate	status	created_at
2026-05-08	19	1	0.0526	completed	2026-05-21 14:03:00

The *predictions* table stores one row per model prediction, as shown in the example below.

Table 4. Example record from the predictions table

prediction_id	batch_date	record_id	biopsy_positive_probability	predicted_label	observed_biopsy_label	model_version	created_at
1	2026-05-08	137	0.642	1	0	sparse_logistic_v1	2026-05-21 21:31:12

The prediction_id is an auto-incrementing internal database identifier for each prediction record. The record_id is a synthetic identifier assigned to each input record because the original UCI dataset does not contain patient IDs. The predictions table also uses a uniqueness constraint on *(batch_date, record_id, model_version)* to avoid duplicate prediction records if inference is rerun for the same batch and model version.

Together, these storage choices support both reproducibility and operational tracking. Flat files make generated outputs easy to inspect, while SQLite makes it easier to audit which batches were processed, retrieve prediction records, and summarize model outputs over time.

7. Serving Predictions

This project uses a batch inference workflow to generate predictions for new data. This design matches the simulated daily ingestion cadence, where new screening records arrive as daily batches rather than as individual real-time events.

After training, the final fitted scikit-learn pipeline is saved as a model file. This saved pipeline includes the preprocessing steps, missingness indicator creation, feature selection, and final logistic regression classifier. During inference, the pipeline is loaded and applied directly to the selected daily batch. This ensures that incoming records are processed in the same way as the historical training data.

For a single daily batch, inference can be run by specifying the arrival date. For example, the script can process records arriving on 2026-05-08. The script filters the future data to the requested date, removes target and metadata columns from the feature matrix, applies the saved pipeline, and generates two outputs for each record: the predicted biopsy-positive probability and the predicted binary label.

The inference script also supports processing all available daily batches. This is useful for running the full simulated workflow and generating predictions for the complete future stream.

Prediction results are saved in two formats. First, each daily batch produces a CSV file under *data/predictions/*, which makes the results easy to inspect and reuse. Second, the same prediction records are written to the SQLite predictions table, together with the batch date, synthetic record ID, model version, predicted probability, predicted label, and observed biopsy label for offline evaluation. The observed biopsy label is not used during inference.

This design provides a simple and reproducible batch prediction workflow: new batch data is selected by date, processed using the saved model pipeline, and stored in both human-readable files and a queryable database.

8. Drift Monitoring

Drift monitoring is included to check whether incoming daily batches differ from the historical data used for model development. In this project, the historical subset is treated as the baseline distribution, and each simulated daily batch is compared against this baseline after ingestion.

The monitoring module checks three types of drift:

1. **Missingness drift**
   For each input feature, the pipeline compares the missing rate in the historical baseline with the missing rate in the daily batch. A drift warning is raised if the absolute difference in missing rate is greater than 0.2. This check is especially relevant for this dataset because some risk-factor and diagnosis-related variables contain missing values.
2. **Numeric feature drift**
   For numeric features, the pipeline compares the mean value in the daily batch with the historical baseline mean. A warning is raised if the absolute difference is greater than two historical standard deviations. This provides a simple rule-based check for large shifts in numeric feature distributions, such as age, duration variables, or count variables.
3. **Prediction score drift**
   The pipeline also monitors the distribution of model outputs. Historical cross-validated prediction probabilities are used as the baseline prediction score distribution. Each daily batch's predicted biopsy-positive probabilities are then compared with this baseline. A warning is raised if the daily batch mean prediction score differs from the baseline mean by more than two baseline standard deviations.

Drift reports are saved under the *reports/* directory for inspection.

In the default simulated daily batches, no major drift was detected. This is expected because the historical and future records were sampled from the same source dataset and the initial split was stratified by the biopsy outcome. Therefore, the default simulation represents a stable incoming data scenario rather than an intentionally shifted deployment scenario.

To verify that the drift monitoring logic can detect a data-quality shift, I also included an optional synthetic stress test. In this stress test, missingness is artificially increased in the following variables: Hormonal Contraceptives, Hormonal Contraceptives (years), IUD, IUD (years), STDs, and STDs (number). This stress test is not treated as real incoming data. It is used only to confirm that the monitoring module raises warnings when a clear missingness shift is introduced.

In a real deployment, drift warnings would not automatically trigger model retraining. Instead, they would first trigger human review and data-quality investigation. For example, a missingness drift warning could indicate a change in the data collection process, or a broken data pipeline. If the shift is persistent and affects model inputs or prediction scores, the next step would be to evaluate model performance when newly labelled outcomes become

available. Recalibration or retraining would then be considered only if the shift is persistent and associated with degraded model performance.

9. Code Structure and Reproducibility

The repository is organized to separate reusable pipeline logic from runnable workflow scripts. The *src/* directory contains the core implementation, while the *scripts/* directory contains scripts for training, inference, drift checking, running the full demo, and the optional stress test.

The main source files are organized as follows:

- *src/config.py* defines shared constants, file paths, random seed, target variables, and pipeline settings.
- *src/data.py* handles data loading, missing value conversion, historical/future splitting, synthetic daily batch assignment, and feature/target separation.
- *src/model.py* defines the preprocessing pipeline and final sparse logistic regression model.
- *src/storage.py* manages the SQLite database, including ingestion logging and prediction storage.
- *src/drift.py* contains the drift monitoring logic.

The *scripts/* directory contains runnable workflows:

- *run_training.py* trains the final model, performs cross-validation, and saves evaluation outputs.
- *run_daily_inference.py* runs prediction for one daily batch or all batches.
- *run_drift_check.py* runs drift checks for one batch or all batches.
- *run_full_demo.py* executes the full end-to-end workflow, including cleaning previous generated outputs, training, inference, storage, and drift monitoring.
- *run_drift_stress_test.py* runs the optional synthetic missingness stress test.

Reproducibility is supported in several ways. A fixed random seed is used wherever applicable. The original dataset is included locally under *data/external/*, so the pipeline does not depend on external network access. Dependencies are listed in *requirements.txt*, and the repository README.md provides instructions for creating a virtual environment and running the workflow.

This structure makes the project easier to inspect, rerun, and extend. The full workflow can be executed with one script, while individual stages can also be run separately for debugging or review. Detailed running instructions are provided in the repository README.md.

10. Limitations

This project is a technical demonstration of an end-to-end machine learning pipeline, not a clinically validated prediction system. Several limitations should be considered.

First, the dataset is small and static. The simulated daily batches are created from a held-out portion of the same dataset rather than from a real continuously updated data source. Therefore, the default incoming batches are expected to be broadly similar to the historical baseline. The drift monitoring module demonstrates how shifts would be checked, but the project does not represent a real deployment environment with changing patient populations or data collection processes.

Second, the positive biopsy class is rare. Only 55 out of 858 records are biopsy-positive. This limits model stability and makes threshold-dependent metrics such as precision, recall, and F1-score sensitive to small changes in the data split. The model comparison and selected features should therefore be interpreted cautiously.

Third, the timing of some variables is not fully clear from the dataset alone. The diagnosis-related variables, namely Dx:Cancer, Dx:CIN, Dx:HPV, and Dx, are treated in this project as historical features available before prediction. In a real screening system, the timing and availability of these variables would need to be verified carefully to avoid temporal leakage.

Fourth, the observed biopsy labels are already available in this static dataset. In a real workflow, biopsy outcomes may arrive later than the initial risk-factor records. The project stores observed labels for offline evaluation, but a real system would need a more explicit delayed-label update process.

Finally, the model has not been externally validated and is not intended for clinical use or clinical-level decision making. Future improvements could include validation on an independent dataset, calibration assessment, threshold selection based on clinical priorities, more formal drift detection methods, and a retraining or recalibration workflow once sufficient newly labelled data becomes available.